



Unit-1

Introduction to Python





Outline

- ✓ Introduction to python
- ✓ Installing python
- ✓ Hello World program using python
- ✓ Data types
- ✓ Variables
- ✓ Expressions
- ✓ Functions
- ✓ String
- ✓ List
- ✓ Tuple
- ✓ Set
- ✓ Dictionary
- ✓ Functions

Introduction to Python

- ❑ Python is an **open source, interpreted, high-level, general-purpose** programming language.
- ❑ Python's design philosophy emphasizes **code readability** with its notable use of significant **whitespace**.
- ❑ Python is **dynamically typed** and **garbage-collected** language.
- ❑ Python was conceived in the late **1980s** as a successor to the **ABC language**.
- ❑ Python was Created by **Guido van Rossum** and first released in **1991**.



Why Python?

□ Python has many advantages

- Easy to learn
- Less code
- Syntax is easier to read
- Open source
- Huge amount of additional open-source libraries

Some libraries listed below.

- **matplotlib** for plotting charts and graphs
- **BeautifulSoup** for HTML parsing and XML
- **NumPy** for scientific computing
- **pandas** for performing data analysis
- **SciPy** for engineering applications, science, and mathematics
- **Scikit** for machine learning
- **Django** for server-side web development
- And many more..

Installing Python

❑ For Windows & Mac:

- ❑ To install python in windows you need to download installable file from <https://www.python.org/downloads/>
- ❑ After downloading the installable file you need to execute the file.

❑ For Linux :

- ❑ For ubuntu 16.10 or newer
 - `sudo apt-get update`
 - `sudo apt-get install python3.11`

❑ To verify the installation

- ❑ Windows :
 - `python --version`
- ❑ Linux :
 - **python3** --version (linux might have python2 already installed, you can check python 2 using **python --version**)

❑ Alternatively we can use anaconda distribution for the python installation

- ❑ <http://anaconda.com/downloads>
- ❑ Anaconda comes with many useful inbuilt libraries.

Hello World using Python

- To write python programs, we can use any text editors or IDE (Integrated Development Environment), Initially we are going to use Visual Studio Code.
- Create new file in editor, save it as **first.py** (Extensions for python programs will be .py)

first.py

```
1 print("Hello World from python")
```

Python line does not end with ;

- To run the python file open command prompt and change directory to where your python file is

```
D:\>cd B.E
D:\B.E>cd 5th
D:\B.E\5th>cd "Phython 2020"
D:\B.E\5th\Phython 2020>cd Demo
```

- Next

```
D:\B.E\5th\Phython 2020\Demo>python first.py
```


Hello World from python

Data types in Python

Name	Type	Description
Data Types		
Integer	<code>int</code>	Whole number such as 0,1,5, -5 etc..
Float	<code>float</code>	Numbers with decimal points such as 1.5, 7.9, -8.2 etc..
String	<code>str</code>	Sequence of character (Ordered) such as “Ankit”, ‘college’, “Surat” etc..
Boolean	<code>bool</code>	Logical values indicating T rue or F alse (T and F here are capital in python)
Data Structures		
List	<code>list</code>	Ordered Sequence of objects, will be represented with square brackets [] Example : [18, “Ankit”, True, 102.3]
Tuple	<code>tup</code>	Ordered immutable sequence of objects, will be represented with round brackets () Example : (18, “Ankit”, True, 102.3)
Set	<code>set</code>	Unordered collection of unique objects, will be represented with the curly brackets { } Example : { 18, “Ankit”, True, 102.3 }
Dictionary	<code>dict</code>	Unordered key : value pair of objects , will be represented with curly brackets { } Example : { “college”: “Ankit”, “code”: “042” }

Variables in Python

- ❑ A Python variable is a reserved memory location to store values.
- ❑ Unlike other programming languages, Python has no command for declaring a variable.
- ❑ A variable is created the moment you first assign a value to it.
- ❑ Python uses **Dynamic Typing** so,
 - ❑ We need not to specify the data types to the variable as it will internally assign the data type to the variable according to the value assigned.
 - ❑ we can also reassign the different data type to the same variable, variable data type will change to new data type automatically.
 - ❑ We can check the current data type of the variable with **type(variablename)** in-built function.
- ❑ Rules for variable name
 - ❑ Name can not start with digit
 - ❑ Space not allowed
 - ❑ Can not contain special character
 - ❑ Python keywords not allowed
 - ❑ **Should** be in lower case

Example of Python variable

□ Example :

demo.py

```
1 x = 10
2 print(x)
3 print(type(x))
4
5 y = 123.456
6 print(y)
7
8 x = "scet"
9 print(x)
10 print(type(x))
```

Reassign same variable to hold different data type

Run in terminal

```
1 python demo.py
```

Output

```
1 10
2 int
3 123.456
4 scet
5 str
```

Python – Input and Output

- ❖ The purpose of a computer is to process data and return results.
- ❖ The data given to the computer is called ‘Input’.
- ❖ The result returned by the computer is called ‘Output’.
- ❖ For performing input-output, Python provides statements
 - Output Statements
 - Input Statements

Output Statements

- ❑ To display output or result, Python provides **print()** function.
- ❑ It has several formats:
- ❑ **print()** – with **display blank line**.
- ❑ **print(“string”)** – the string is displayed as it is.
- ❑ We can use escape sequence characters inside **print()** function.
- ❑ We can use **repetition operator (*)** inside **print()** function.
- ❑ **To join two string** in display, we can use **‘+’** operator inside **print()** function.

Output Statements

□ print() function

```
1 print("Ankit")
2 print("Scet")
3 print("First line \n Second line")
4 print("this is \t tab sequence")
5 print("hello"*3, "Surat"*2)
6 print("city name="+ "Surat")
7 a=10,b=20;
8 print(a,b)
9 print("Number 1 =",a,"Number 2=",b)
10 print("Hello","Student",sep=",")
```

Output : *Ankit*

Output : *Scet*

Output : *First line*
Second line

Output : *this is tab sequence*

Output : *hellohellohello SuratSurat*

Output : *city name=Surat*

Output : *10 20*

Output : *Number 1 = 10 Number 2= 20*

Output : *Hello, Student*

Input Statements

- ❑ To accept input from keyboard, Python provides **input()** function.
- ❑ This function **takes a value from the keyboard and returns it as a string**
- ❑ Once the value comes into variable, it can be converted into 'int' or 'float'.

```
1 a=input("Enter the value")
2 b=input("Enter the value")
3 print(a)
4 print(b)
5 print(a+b)
```

Output :

Enter the value 10
Enter the value 20
10
20
1020

```
1 a=int(input("Enter the value"))
2 b=int(input("Enter the value"))
3 print(a)
4 print(b)
5 print(a+b)
```

Output :

Enter the value 10
Enter the value 20
10
20
30

Python Operators

- ❑ Operators are used to perform operations on variables and values.
- ❑ Python divides the operators in the following groups:
- ❑ Arithmetic operators
- ❑ Assignment operators
- ❑ Comparison operators
- ❑ Logical operators
- ❑ Identity operators
- ❑ Membership operators
- ❑ Bitwise operators

Arithmetic Operators

□ Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

❑ Example of Arithmetic Operators in Python

```
a = 15
b = 4

# Addition
print("Addition:", a + b)

# Subtraction
print("Subtraction:", a - b)

# Multiplication
print("Multiplication:", a * b)

# Division
print("Division:", a / b)

# Floor Division
print("Floor Division:", a // b)

# Modulus
print("Modulus:", a % b)

# Exponentiation
print("Exponentiation:", a ** b)
```

Output

```
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Floor Division: 3
Modulus: 3
Exponentiation: 50625
```


Assignment Operators

□ Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	<code>x = 5</code>	<code>x = 5</code>
+=	<code>x += 3</code>	<code>x = x + 3</code>
-=	<code>x -= 3</code>	<code>x = x - 3</code>
*=	<code>x *= 3</code>	<code>x = x * 3</code>
/=	<code>x /= 3</code>	<code>x = x / 3</code>

❑ Example of Assignment Operators in Python:

```
a = 10  
b = a  
print(b)  
b += a  
print(b)  
b -= a  
print(b)  
b *= a  
print(b)  
b <<= a  
print(b)
```

Output

```
10  
20  
10  
100  
102400
```

Comparison Operators (Relational Operators)

□ Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>

❑ Example of Comparison Operators in Python

```
a = 13
b = 33

print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

Output

```
False
True
False
True
False
True
```



Logical Operators

□ Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

❑ Example of Logical Operators in Python:

```
a = True  
b = False  
print(a and b)  
print(a or b)  
print(not a)
```

Output

```
False  
True  
False
```

Bitwise Operators

□ Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to 1 if both bits are 1	$x \& y$
	OR	Sets each bit to 1 if one of two bits is 1	$x y$
^	XOR	Sets each bit to 1 if only one of two bits is 1	$x \wedge y$
~	NOT	Inverts all the bits	$\sim x$
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	$x \ll 2$
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	$x \gg 2$

❑ Example of Bitwise Operators in Python:

```
a = 10
```

```
b = 4
```

```
print(a & b)
```

```
print(a | b)
```

```
print(~a)
```

```
print(a ^ b)
```

```
print(a >> 2)
```

```
print(a << 2)
```

Output

0

14

-11

14

2

40

Identity Operators

- ❑ In Python, **is** and **is not** are the identity operators both are used to check if two values are located on the same part of the memory.
- ❑ Two variables that are equal do not imply that they are identical.

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

```
x = ["apple", "banana"]  
y = ["apple", "banana"]  
z = x
```

```
print(x is z)
```

```
print(x is y)
```

```
print(x == y)
```

```
True  
False  
True
```

```
a = 10
```

```
b = 20
```

```
c = a
```

```
print(a is not b)
```

```
print(a is c)
```

```
True  
True
```

Membership Operators

- ❑ In Python, **in** and **not in** are the membership operators that are used to test whether a value or variable is in a sequence.
- ❑ **in** *True if value is found in the sequence*
not in *True if value is not found in the sequence*
- ❑ Examples

```
x = ["apple", "banana"]  
print("banana" in x)
```

```
x = ["apple", "banana"]  
print("pineapple" not in x)
```

```
x = 24
y = 20
list = [10, 20, 30, 40, 50]

if (x not in list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")
|
if (y in list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Output

```
x is NOT present in given list
y is present in given list
```

Operator Precedence

□ Precedence represents the priority level.

Precedence	Operator Sign	Operator Name
Highest	**	Exponentiation
	+X, -X, ~X	Unary positive, unary negative, bitwise negation
	*, /, //, %	Multiplication, division, floor, division, modulus
	+, -	Addition, subtraction
	<<, >>	Left-shift, right-shift
	&	Bitwise AND
	^	Bitwise XOR
		Bitwise OR
	==, !=, <, <=, >, >=, is, is not	Comparison, Identity
	not	Boolean NOT
	and	Boolean AND
Lowest	or	Boolean OR

Python Data Types

- ❖ In Python, **data types** define the kind of value a variable can hold and determine what operations can be performed on it.
- ❖ Python is **dynamically typed**, meaning you don't need to declare the type of a variable explicitly — it is determined at runtime based on the assigned value.
- ❖ Python provides a rich set of built-in data types, which can be categorized as:
 - ❖ Numeric Types – int, float, complex
 - ❖ Sequence Types – str, list, tuple
 - ❖ Boolean Type – bool
 - ❖ Set Types – set, frozenset
 - ❖ Mapping Type – dict

1. Numeric Data Types in Python

- ❑ The numeric data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, or even a complex number.
- ❑ **Integers** - This value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). In Python, there is no limit to how long an integer value can be.
- ❑ **Float** - This value is represented by the float class. It is a real number with a floating-point representation. It is specified by a decimal point.
- ❑ **Complex Numbers** - A complex number is represented by a complex class. It is specified as $(\text{real part}) + (\text{imaginary part})i$

```
a = 5
print(type(a))

b = 5.0
print(type(b))

c = 2 + 4j
print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Sequence Data Types in Python

- ❑ The sequence Data Type in Python is the ordered collection of similar or different Python data types.
- ❑ Sequences allow storing of multiple values in an organized and efficient fashion.
- ❑ Python String
- ❑ Python List
- ❑ Python Tuple
- ❑ String example ❑

```
s = 'Welcome to the Python World'
print(s)
```

```
# check data type
print(type(s))
```

```
# access string with index
print(s[0])
print(s[2])
print(s[-1])
```

```
Welcome to the Python World
<class 'str'>
W
l
d
```


□ List Data Type

- Lists are just like arrays, declared in other languages which is an ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

```
a = ["Hello", "Students", "Goodday"]  
print("Accessing element from the list")  
print(a[0])  
print(a[2])  
  
print("Accessing element using negative indexing")  
print(a[-1])  
print(a[-3])
```

```
Accessing element from the list  
Hello  
Goodday  
Accessing element using negative indexing  
Goodday  
Hello
```

□ Tuple Data Type

- Just like a list, a tuple is also an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

```
# initiate empty tuple  
tup1 = ()
```

```
tup2 = ('Hello', 'Students')  
print("\nTuple with the use of String: ", tup2)
```

Tuple with the use of String: ('Hello', 'Students')

```
tup1 = tuple([1, 2, 3, 4, 5])
```

```
# access tuple items  
print(tup1[0])  
print(tup1[-1])  
print(tup1[-3])
```

1
5
3

3. Boolean Data Type in Python

❖ Python Data type with one of the two built-in values, True or False.

```
print(type(True))  
print(type(False))  
print(type(true))
```

```
<class 'bool'>  
<class 'bool'>
```

```
Traceback (most recent call last):  
  File "/home/7e8862763fb66153d70824099d4f5fb7.py", line 8, in  
    print(type(true))  
NameError: name 'true' is not defined
```

4. Set Data Type in Python

In Python Data Types, Set is an unordered collection of data types that is mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

SET Example

```
s1 = {"a","b","c","c","d","a"}  
print("Set with the use of String: ", s1)
```

```
s2 = {"hello", "For", "hello"}  
print("Set with the use of List: ", s2)
```

```
Set with the use of String: {'b', 'a', 'c', 'd'}  
Set with the use of List: {'For', 'hello'}
```

- ❖ **frozenset() in Python**
- ❖ **Python Method** creates an immutable Set object.
- ❖ It is a built-in Python function. As it is a set object, therefore, we cannot have duplicate values in the frozenset.

```
animals = frozenset(["cat", "dog", "lion"])  
print("cat" in animals)  
print("elephant" in animals)
```

```
True  
False
```

5. Mapping type: Dictionary Data Type

- ❖ A dictionary in Python is a collection of data values, used to store data values like a map, unlike other Python Data Types that hold only a single value as an element, a Dictionary holds a key: value pair.
- ❖ Key-value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

```
d = {1: 'Hello', 'name': 'Students', 3: 'Good day'}
```

```
# Accessing an element using key
```

```
print(d['name'])
```

```
# Accessing a element using get
```

```
print(d.get(3))
```

Students

Good day

Literals in Python

- ❑ **Literals** in Python are fixed values written directly in the code that represent constant data. They provide a way to store numbers, text, or other essential information that does not change during program execution.
- ❑ Python supports different types of literals, such as numeric literals, string literals, Boolean literals

```
# Integer literals
```

```
a = 100
```

```
b = -50
```

```
# Floating-point literals
```

```
c = 3.14
```

```
d = -0.005
```

```
# Complex number literals
```

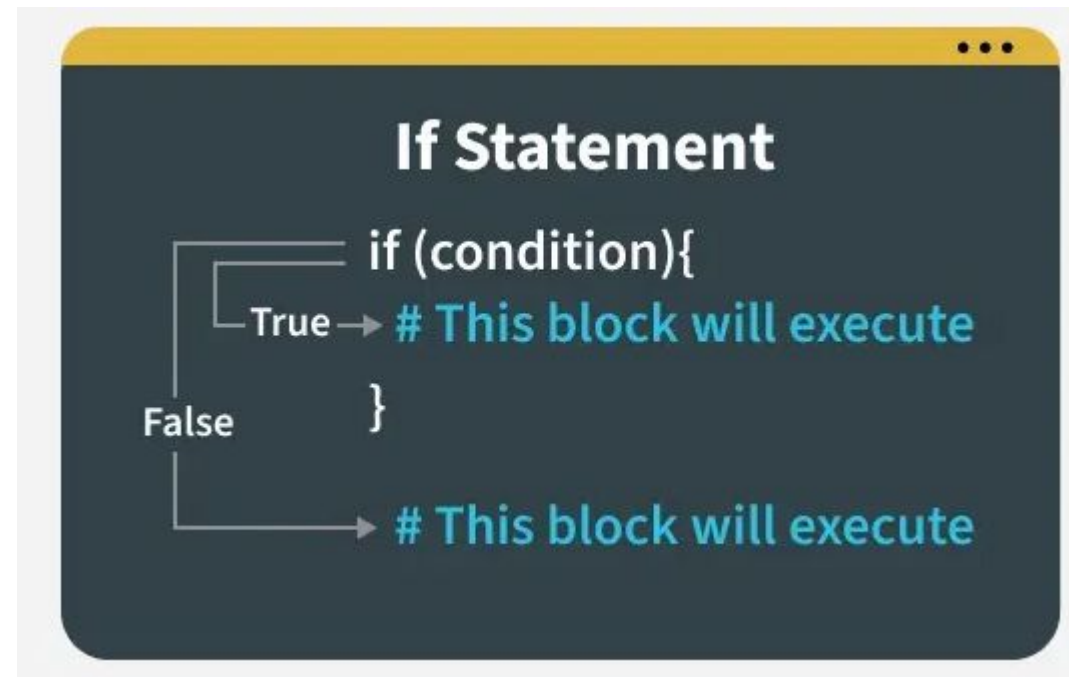
```
e = 4 + 7j
```

```
print(a, b, c, d, e)
```

```
100 -50 3.14 -0.005 (4+7j)
```

Python If Else Statements - Conditional Statements

- ❖ In Python, If-Else is a fundamental conditional statement used for decision-making in programming. If...Else statement allows to execution of specific blocks of code depending on the condition is True or False.
- ❖ if Statement : if statement is the most simple decision-making statement. If the condition evaluates to True, the block of code inside the if statement is executed.



if Statement

- ▶ if statement is written using the if keyword followed by condition and colon(:) .
- ▶ Code to execute when the condition is true will be ideally written in the next line with Indentation (white space).
- ▶ Python relies on indentation to define scope in the code (Other programming languages often use curly-brackets for this purpose).

Syntax

```
if some_condition :  
    # Code to execute when condition is true
```

if statement ends with :

1
2

Indentation (tab/whitespace) at the beginning

ifdemo.py

```
1 x = 10  
2  
3 if x > 5 :  
4     print("X is greater than 5")
```

Run in terminal

```
1 python ifdemo.py
```

Output

```
1 X is greater than 5
```

Example of if statement

```
a = 33
b = 200

if b > a:
    print("b is greater than a")
```

b is greater than a

Without indentation

```
a = 33
b = 200

if b > a:
print("b is greater than a")
```

```
File "<frozen importlib._bootstrap_external>", line 1059, in source_to_code
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "./prog.py", line 5
    print("b is greater than a")
    ^
IndentationError: expected an indented block after 'if' statement on line 4
```

example 2

```
i = 10
```

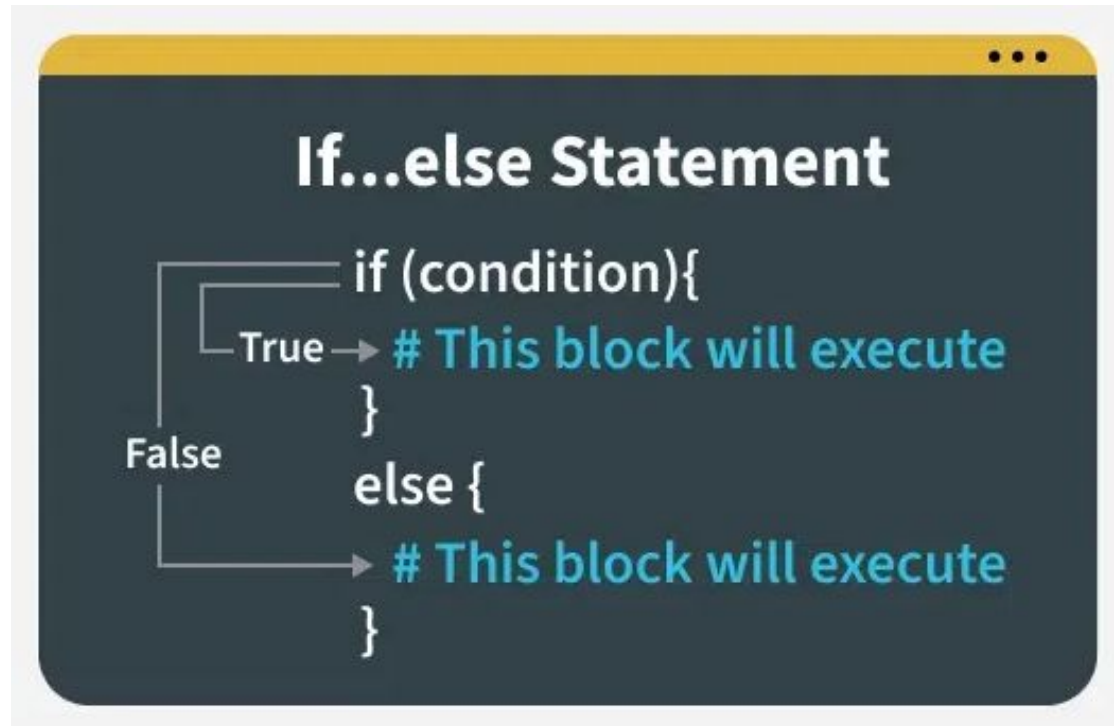
```
# Checking if i is greater than 15
if (i > 15):
    print("10 is less than 15")

print("I am Not in if")
```

```
I am Not in if
```

if....else Statement

- ❖ if...else statement is a control statement that helps in decision-making based on specific conditions.
- ❖ If the condition in the if statement is not true, the else block will be executed.



If else statement

Syntax

```
1 if some_condition :  
2     # Code to execute when condition is true  
3 else :  
4     # Code to execute when condition is false
```

ifelsedemo.py

```
1 x = 3  
2  
3 if x > 5 :  
4     print("X is greater than 5")  
5 else :  
6     print("X is less than 5")
```

Run in terminal

```
1 python ifelsedemo.py
```

Output

```
1 X is less than 5
```

❖ If Else in One-line

❖ If we need to execute a single statement inside the if or else block then one-line shorthand can be used.

```
a = -2
```

```
# Ternary conditional to check if number is positive or negative  
res = "Positive" if a >= 0 else "Negative"  
print(res)
```

Output

Negative

❖ Logical Operators with If..Else

```
age = 25
```

```
exp = 10
```

```
# Using '>' operator & 'and' with if-else  
if age > 23 and exp > 8:  
    print("Eligible.")  
else:  
    print("Not eligible.")
```

Output

Eligible.

if-elif-else

- ❖ if-elif-else statement in Python is used for multi-way decision-making.
- ❖ This allows us to check multiple conditions sequentially and execute a specific block of code when a condition is True.
- ❖ If none of the conditions are true, the else block is executed.

```
i = 25

# Checking if i is equal to 10
if (i == 10):
    print("i is 10")
# Checking if i is equal to 15

elif (i == 15):
    print("i is 15")
# Checking if i is equal to 20

elif (i == 20):
    print("i is 20")

# If none of the above conditions are true
else:
    print("i is not present")
```

i is not present

If, elif and else statement

Syntax

```
1 if some_condition_1 :  
2     # Code to execute when condition 1 is true  
3 elif some_condition_2 :  
4     # Code to execute when condition 2 is true  
5 else :  
6     # Code to execute when both conditions are false
```

Else if remove se

ifelifdemo.py

```
1 x = 10  
2  
3 if x > 12 :  
4     print("X is greater than 12")  
5 elif x > 5 :  
6     print("X is greater than 5")  
7 else :  
8     print("X is less than 5")
```

Run in terminal

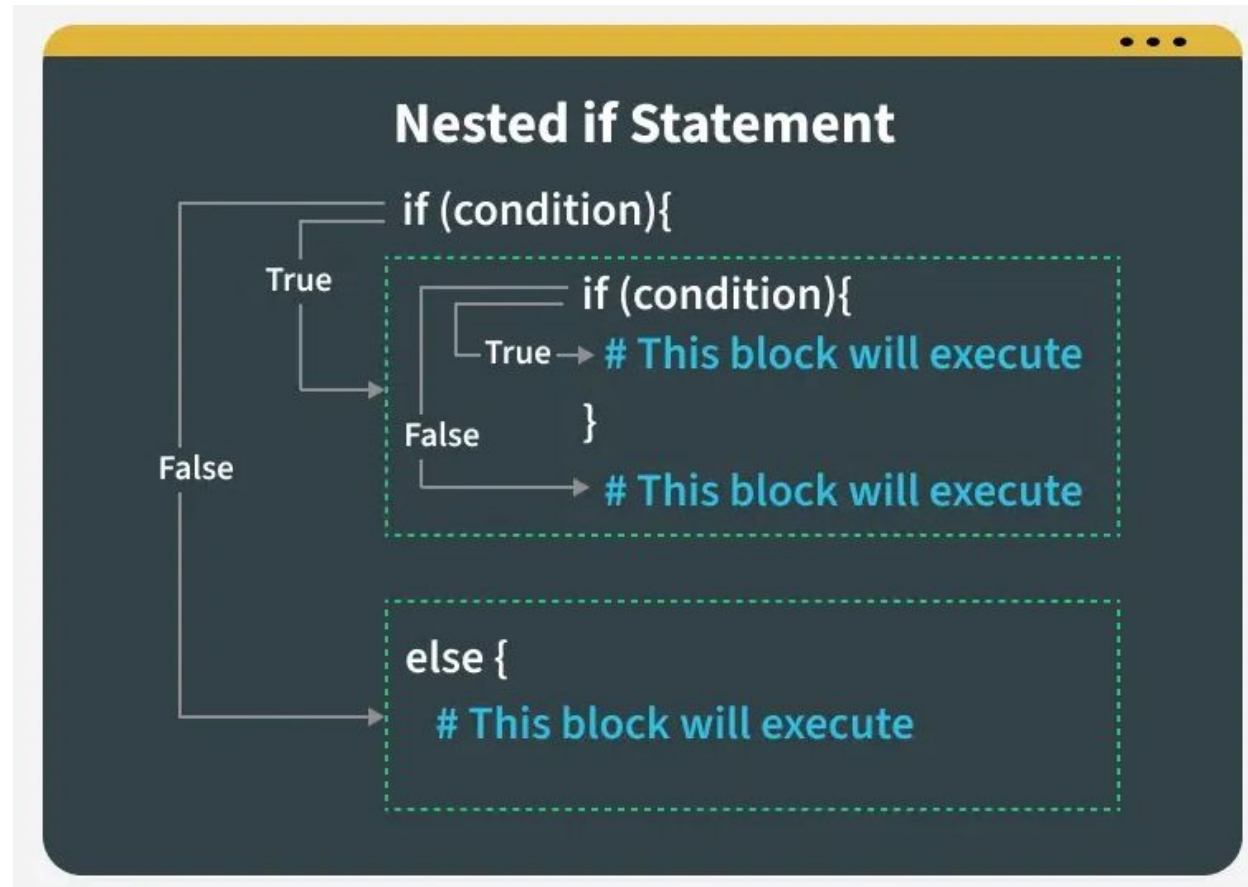
```
1 python ifelifdemo.py
```

Output

```
1 X is greater than 5
```


Nested If Else Statement

- ❖ Nested if...else statement occurs when if...else structure is placed inside another if or else block.
- ❖ Nested If..else allows the execution of specific code blocks based on a series of conditional checks.



Nesteds if statement

```
1 name = "Alice"
2 score = 78
3 if name == "Alice":
4     if score >= 80:
5         print("Great job, Alice! You got an A.")
6     else:
7         print("Good effort, Alice! Keep it up.")
8 else:
9     print("You're doing well, but this message is for Alice.")
```

Output

1

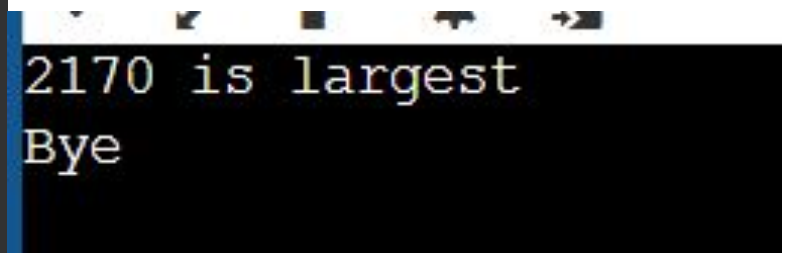
Good effort, Alice! Keep it up.

Python program to find largest of three numbers

```
1  a=140
2  b=260
3  c=2170
4
5  if a>b:
6      if a>c:
7          print(a,"is largest")
8      else :
9          print(c,"is largest")
10 else:
11     if b>c:
12         print(b,"is largest")
13     else:
14         print(c,"is largest")
15
16 print("Bye")
```

Python program to find largest of three numbers

```
1  a=140
2  b=260
3  c=2170
4
5  if a>b:
6      if a>c:
7          print(a,"is largest")
8      else :
9          print(c,"is largest")
10 else:
11     if b>c:
12         print(b,"is largest")
13     else:
14         print(c,"is largest")
15
16 print("Bye")
```



2170 is largest
Bye

Python program to find largest of three numbers

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))

if (num1 >= num2) and (num1 >= num3):
    largest = num1
elif (num2 >= num1) and (num2 >= num3):
    largest = num2
else:
    largest = num3

print("The largest number is:", largest)
```


Python program to find largest of three numbers

```
num1 = float(input("Enter first number: "))  
num2 = float(input("Enter second number: "))  
num3 = float(input("Enter third number: "))
```

```
if (num1 >= num2) and (num1 >= num3):  
    largest = num1  
elif (num2 >= num1) and (num2 >= num3):  
    largest = num2  
else:  
    largest = num3
```

```
print("The largest number is:", largest)
```

```
Enter first number: 78  
Enter second number: 89  
Enter third number: 966  
The largest number is: 966.0
```

Python program to find largest of three numbers

```
1 num1 = float(input("Enter first number: "))
2 num2 = float(input("Enter second number: "))
3 num3 = float(input("Enter third number: "))
4
5 largest = max(num1, num2, num3)
6
7 print("The largest number is:", largest)
```

Range()

- ❑ The range() object is also known as range() function in python is useful to provide a sequence of numbers.
- ❑ range(n) gives numbers from 0 to n-1.
- ❑ If we write range(10), It will give number from 0 to 9 in sequence.
- ❑ If we write range(5,10), it will give number from 5 to 9
- ❑ If we write range(1,10,2), it will give number 1,3,5,7,9

forrange.py

```
1 for i in range(1,5):  
2     print(i)
```

Output :

1
2
3
4

forrange1.py

```
1 for i in range(1,5,2):  
2     print(i)
```

Output :

1
3

Key characteristics of range():

- It only works with integers; floating-point numbers are not supported.
- The stop value is always exclusive, meaning it is not included in the generated sequence.
- range() objects are memory-efficient as they only store the start, stop, and step values, generating numbers on demand during iteration, rather than creating a full list in memory.
- The step value must not be zero.

Syntax of Python range() function

Syntax: range(start, stop, step)

Parameter :

start: [optional] start value of the sequence

stop: next value after the end value of the sequence

step: [optional] integer value, denoting the difference between any two numbers in the sequence

For loop

- ❖ A for loop in Python is used to iterate over a sequence (like a list, tuple, string, or range). It allows you to execute a block of code multiple times—once for each item in the sequence.

```
for variable in sequence:  
    # block of code (loop body)
```

- ❖ variable: a temporary name assigned to each item in the sequence as the loop iterates.
- ❖ sequence: a collection of items (e.g., list, string, range).
- ❖ Indentation is essential to define the block of code inside the loop.

For loop in python

- Many objects in python are **iterable**, meaning we can iterate over every element in the object.
 - such as every elements from the List, every characters from the string etc..
- We can use for loop to execute block of code for each element of iterable object.

Syntax

```
1 for temp_item in iterable_object :  
2     # Code to execute for each object in iterable
```

For loop ends with :

Indentation (tab/whitespace) at the beginning

fordemo1.py

```
1 my_list = [1, 2, 3, 4]  
2 for list_item in my_list:  
3     print(list_item)
```

Output :

1
2
3
4

fordemo2.py

```
1 my_list = [1,2,3,4,5,6,7,8,9]  
2 for list_item in my_list :  
3     if list_item % 2 == 0 :  
4         print(list_item)
```

Output :

2
4
6
8

Example

```
1 for char in "Python":  
2     print(char)
```

Example

```
1 for char in "Python":  
2     print(char)
```

```
P  
y  
t  
h  
o  
n
```

Problem Statements

1. Write a python program to print 1 to 10.
2. write a python program to print 1 to n.
3. write a python program to print all the odd numbers between 1 to N.
4. write a python program to print all the even numbers between 1 to N.
5. Write a python program to Calculate the Sum of First N Natural Numbers.
6. Write a python program to Calculate the Sum of First N odd Numbers.
7. Write a python program to Calculate the Sum of First N even Numbers.
8. Write a python program to print multiplication table of a number.

Problem Statements-Solutions

```
1 for i in range(1,10):  
2     print(i)
```

```
1 n=int(input("Enter the n"))  
2 for i in range(1,n):  
3     print(i)
```

```
1 n=int(input("Enter the n"))  
2 for i in range(1,n):  
3     if(i%2==0):  
4         print(i)  
5
```


Problem Statements-Solutions

```
1 n=int(input("Enter the n"))
2 for i in range(1,n):
3     if(i%2==0):
4         print(i)
5
```

```
1 n=int(input("Enter the n"))
2 sum=0
3 for i in range(1,n+1):
4     sum+=i
5 print("Sum is:",sum)
```




Exercise

1. Write a python program to print Sum of Digits in a String.

Exercise

1. Write a python program to print Sum of Digits in a String.

```
1  a=input("enter the number")
2  sum=0
3  for i in a:
4      i=int(i)
5      sum+=i
6  print("Sum of digit is:",sum)
```

Exercise

From a numbers 1 to 50, print numbers divisible by both 3 and 5 using a for loop.

Exercise

From a numbers 1 to 50, print numbers divisible by both 3 and 5 using a for loop.

```
1 for i in range(1, 51):  
2     if i % 3 == 0 and i % 5 == 0:  
3         print(i)  
4
```

Write a python program to accept N numbers from user and display sum and average

```
1 num = int(input('How many numbers: '))
2 total_sum = 0
3
4 for n in range(num):
5     numbers = float(input('Enter number : '))
6     total_sum += numbers
7
8
9 avg = total_sum/num
10 print('total of ', num, ' numbers is :', total_sum)
11 print('Average of ', num, ' numbers is :', avg)
```

Extra Practicals

1. Break on First Negative Given a list of integers, print each number until the first negative number is found (then stop using break).
2. Find the Largest Number in a List. Given a list of integers, use a for loop to find and print the largest number.
- 3.

Extra practical Solution

```
1  nums = [5, 12, 8, 20, 3]
2  max_num = nums[0]
3  for num in nums:
4      if num > max_num:
5          max_num = num
6  print("Largest:", max_num)
7
```

```
1  nums = [10, 20, 5, -3, 8]
2  for num in nums:
3      if num < 0:
4          break
5      print(num)
6
```

While loop

- While loop will continue to execute block of code until some condition remains True.

Syntax

```
1 while some_condition :  
2     # Code to execute in loop
```

while loop ends with :

Indentation (tab/whitespace) at the beginning

whiledemo.py

```
1 x = 0  
2 while x < 3 :  
3     print(x)  
4     x += 1 # x++ is valid in python
```

Output :

0
1
2

withelse.py

```
1 x = 5  
2 while x < 3 :  
3     print(x)  
4     x += 1 # x++ is valid in python  
5 else :  
6     print("X is greater than 3")
```

Output :
X is greater
than 3

break, continue & pass keywords

❑ **break** : Breaks out of the current closest enclosing loop.

❑ **continue** : Goes to the top of the current closest enclosing loop.

❑ **Pass** : Does nothing at all, will be used as a placeholder in conditions where you don't want to write anything.

breakdemo.py

```
1 for temp in range(5) :  
2     if temp == 2 :  
3         break  
4  
5     print(temp)
```

Output :
0
1

continuedemo.py

```
1 for temp in range(5) :  
2     if temp == 2 :  
3         continue  
4  
5     print(temp)
```

Output :
0
1
3
4

passdemo.py

```
1 for temp in range(5) :  
2     pass
```

Output : (nothing)

Problem Statements

```
s="Welcome to world of Python"  
print(s)  
print(s[5])  
print(s[-1:-5:-1])  
print(s[2]*3)
```

```
print(s): Welcome to world of Python  
print(s[5]): m  
print(s[-1:-5:-1]) : noht  
print(s[2]*3): ll
```

```
s="SCET is the best college"  
print(s)  
print(s[15::1])  
print(s[-1:-5:-1])  
print(s[-10:-1:2])
```

Input Statements

□ Conversion of a number in any base to Decimal is also possible with **int() casting function**.

input1.py

```
1 bin=input("Enter the number")
2 a=int(bin,2)
3 print("number=",a)
4 oct=input("Enter the number")
5 b=int(oct,2)
6 print("number=",b)
```

Output :

Enter the number100
number= 4

Enter the number1001
number= 9

Pattern in Python

```
1 n=int(input('enter row'))
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print(j,end='')
5     print()
6
```

```
enter row5
1
12
123
1234
12345
```

```
1 for i in range(1,6):  
2     for k in range(1,6-i):  
3         print(" ",end=" ")  
4     for j in range(1,i+1):  
5         print("* ",end='')  
6     print()  
7
```

```
      *  
     * *  
    * * *  
   * * * *  
  * * * * *  
 * * * * *
```

Array

- ❑ `import array as arr`
- ❑ `variablename = array(typecode,[elements])`
- ❑ `variablename` would be the name of the array.
- ❑ The **typecode** specifies what kind of elements would be stored in the array. Whether it would be an array of integers, an array of floats or an array of any other Python data type.
- ❑ Remember that all elements should be of the same data type.
- ❑ Inside square brackets you mention the elements that would be stored in the array, with each element being separated by a comma.
- ❑ You can also create an empty array by just writing `variablename = array(typecode)` alone, without any elements.

Arrays

```
import array as arr

numbers = arr.array('i', [10, 20, 30])

print(numbers)

#output

#array('i', [10, 20, 30])
```

TYPECODE	C TYPE	PYTHON TYPE	SIZE
'b'	signed char	int	1
'B'	unsigned char	int	1
'u'	wchar_t	Unicode character	2
'h'	signed short	int	2
'H'	unsigned short	int	2
'i'	signed int	int	2
'I'	unsigned int	int	2
'l'	signed long	int	4
'L'	unsigned long	int	4
'q'	signed long long	int	8
'Q'	unsigned long long	int	8
'f'	float	float	4
'd'	double	float	8

Float Array

```
from array import *  
  
#an array of floating point values  
numbers = array('d',[10.0,20.0,30.0])  
  
print(numbers)  
  
#output  
  
#array('d', [10.0, 20.0, 30.0])
```




Length of array

```
import array as arr

numbers = arr.array('i', [10, 20, 30])

print(len(numbers))

#output
# 3
```

Access each individual element in an array

```
import array as arr

numbers = arr.array('i', [10, 20, 30])

print(numbers[0]) # gets the 1st element
print(numbers[1]) # gets the 2nd element
print(numbers[2]) # gets the 3rd element

#output

#10
#20
#30
```

How to Search Through an Array in Python

```
import array as arr

numbers = arr.array('i',[10,20,30])

#search for the index of the value 10
print(numbers.index(10))

#output

#0
```

```
import array as arr
```

```
numbers = arr.array('i',[10,20,30,10,20,30])
```

```
#search for the index of the value 10
```

```
#will return the index number of the first instance of the value 10
```

```
print(numbers.index(10))
```

```
#output
```

```
#0
```

Print array element one by one

```
import array as arr

numbers = arr.array('i',[10,20,30])

for number in numbers:
    print(number)
```

#output

#10

#20

#30

Methods For Performing Operations on Arrays in Python

- `Append()`
- `Extend()`
- `Insert()`
- `Remove()`
- `Pop()`

How to Change the Value of an Item in an Array

```
import array as arr

#original array
numbers = arr.array('i',[10,20,30])

#change the first element
#change it from having a value of 10 to having a value of 40
numbers[0] = 40

print(numbers)

#output

#array('i', [40, 20, 30])
```

How to Add a New Value to an Array

```
import array as arr

#original array
numbers = arr.array('i',[10,20,30])

#add the integer 40 to the end of numbers
numbers.append(40)

print(numbers)

#output

#array('i', [10, 20, 30, 40])
```


How to Add a multiple Value to an Array

```
import array as arr

#original array
numbers = arr.array('i',[10,20,30])

#add the integers 40,50,60 to the end of numbers
#The numbers need to be enclosed in square brackets

numbers.extend([40,50,60])

print(numbers)

#output

#array('i', [10, 20, 30, 40, 50, 60])
```

To add an item at a specific position

```
import array as arr

#original array
numbers = arr.array('i',[10,20,30])

#add the integer 40 in the first position
#remember indexing starts at 0

numbers.insert(0,40)

print(numbers)

#output

#array('i', [40, 10, 20, 30])
```

How to Remove a Value from an Array

- ❑ With `remove()`, only the first instance of the value you pass as an argument will be removed.

```
import array as arr

#original array
numbers = arr.array('i', [10, 20, 30])

numbers.remove(10)

print(numbers)

#output
#array('i', [20, 30])
```

Pop

- Use the pop() method, and specify the position of the element to be removed.

```
import array as arr

#original array
numbers = arr.array('i', [10, 20, 30, 10, 20])

#remove the first instance of 10
numbers.pop(0)

print(numbers)

#output

#array('i', [20, 30, 10, 20])
```

Slicing in an Array

```
import array as arr
l = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

a = arr.array('i', l)
print("Initial Array: ")
for i in (a):
    print(i, end=" ")
Sliced_array = a[3:8]
print("\nSlicing elements in a range 3-8: ")
print(Sliced_array)
Sliced_array = a[5:]
print("\nElements sliced from 5th "
      "element till the end: ")
print(Sliced_array)
Sliced_array = a[:]
print("\nPrinting all elements using slice operation: ")
print(Sliced_array)
```

String in python

- ❑ String is **Ordered Sequence of character** such as “Ankit”, ‘SCET’, “Surat” etc..
- ❑ Strings are **arrays of bytes** representing **Unicode** characters.
- ❑ String can be represented as single, double or triple quotes.
- ❑ String with **triple** Quotes allows **multiple** lines.
- ❑ String in python is **immutable**.
- ❑ Square brackets can be used to access elements of the string, Ex. “ABCDEFGH”[1] = B, characters can also be accessed with reverse index like “ABCDEFGH”[-1] = H.

String index

x	=	"	A	B	C	D	E	F	G	"
index	=		0	1	2	3	4	5	6	
Reverse index	=		-7	-6	-5	-4	-3	-2	-1	

String functions in python

□ Python has lots of built-in methods that you can use on strings, we are going to cover some frequently used methods for string like

- **len()**
- count()
- capitalize(), lower(), upper()
- istitle(), islower(), isupper()
- find(), rfind(), replace()
- index(), rindex()
- Methods for validations like
 - isalpha(), isalnum(), isdecimal(), isdigit()
- strip(), lstrip(), rstrip()
- Etc..

□ **Note :** len() is not the method of the string but can be used to get the length of the string

lendemo.py

```
1 x = "Ankit"  
2 print(len(x))
```

Output : 5 (length of "Ankit")

String methods (cont.)

- ❑ **count()** method will returns the number of times a specified value occurs in a string.

countdemo.py

```
1 x = "ankita"  
2 ca = x.count('a')  
3 print(ca)
```

Output : 2 (occurrence of 'a' in "Ankita")

- ❑ **title()**, **lower()**, **upper()** will returns capitalized, lower case and upper case string respectively.

change-case.py

```
1 x = "SCET College"  
2 c = x.title()  
3 l = x.lower()  
4 u = x.upper()  
5 print(c)  
6 print(l)  
7 print(u)
```

Output : Scet College

Output : scet college

Output : SCET COLLEGE

String methods (cont.)

- ❑ **istitle(), islower(), isupper()** will returns True if the given string is capitalized, lower case and upper case respectively.

checkcase.py

```
1 x = 'scet college'
2 c = x.istitle()
3 l = x.islower()
4 u = x.isupper()
5 print(c)
6 print(l)
7 print(u)
```

Output : *False*

Output : *True*

Output : *False*

- ❑ **strip()** method will remove whitespaces from both side of the string and returns the string.

stripdemo.py

```
1 x = '    Ankit    '
2 f = x.strip()
3 print(f)
```

Output : Ankit(without space)

- ❑ **rstrip()** and **lstrip()** will remove whitespaces from right and left side respectively.

String methods (cont.)

- ❑ **find()** method will search the string and returns the index at which they find the specified

finddemo.py

```
1 x = 'SCET institute, Surat, india'  
2 f = x.find('in')  
3 print(f)
```

Output : 5 (occurrence of 'in' in x)

- ❑ **rfind()** method will search the string and returns the last index at which they find the specified value

rfinddemo.py

```
1 x = 'SCET institute, Surat, india'  
2 r = x.rfind('in')  
3 print(r)
```

Output : 23 (last occurrence of 'in' in x)

- ❑ **Note :** **find()** and **rfind()** will **return -1** if they are unable to find the given string.

- ❑ **replace()** method will replace str1 with str2 from our string and return the updated string

```
1 x = 'SCET institute, Surat, india'  
2 r = x.replace('india', 'INDIA')  
3 print(r)
```

Output :
"SCET institute, Surat, INDIA"

String methods (cont.)

- ❑ **index()** method will search the string and returns the index at which they find the specified value, but if they are unable to find the string it will raise an exception.

indexdemo.py

```
1 x = 'SCET institute, Surat, india '  
2 f = x.index('in')  
3 print(f)
```

Output : 5 (occurrence of 'in' in x)

- ❑ **rindex()** will search the string and returns the last index at which they find the specified value, but if they are unable to find the string it will raise an exception.

rindexdemo.py

```
1 x = 'SCET institute, Surat, india '  
2 r = x.rindex('in')  
3 print(r)
```

Output : 23 (last occurrence of 'in' in x)

- ❑ Note : **find()** and **index()** are almost same, the only difference is if **find()** is unable to find the string it will return -1 and if **index()** is unable to find the string it will raise an exception.

String methods (cont.)

- ❑ **isalnum()** method will return true if all the characters in the string are alphanumeric (i.e either alphabets or numeric).

isalnumdemo.py

```
1 x = 'ankit123'  
2 f = x.isalnum()  
3 print(f)
```

Output : *True*

- ❑ **isalpha()** and **isnumeric()** will return true if all the characters in the string are only alphabets and numeric respectively.
- ❑ **isdecimal()** will return true if all the characters in the string are decimal.

isdecimaldemo.py

```
1 x = '123.5'  
2 r = x.isdecimal()  
3 print(r)
```

Output : *True*

- ❑ **Note :** **isnumeric()** and **isdigit()** are almost same, you suppose to find the difference as Home work assignment for the string methods.

String Slicing

- We can get the substring in python using string slicing, we can specify start index, end index and steps (colon separated) to slice the string.

syntax

```
x = 'Sarvajanik College of Engineering and Technology, Surat, gujarat, INDIA'  
subx = x[startindex:endindex:steps]
```

endindex will not be included in the substring

strslicedemo.py

```
1 x = 'Sarvajanik College of Engineering and Technology,  
2 Surat, gujarat, INDIA'  
3 subx1 = x[0:10]  
4 subx2 = x[50:55]  
5 subx3 = x[66:]  
6 subx4 = x[::2]  
7 subx5 = x[::-1]  
8 print(subx1)  
9 print(subx2)  
10 print(subx3)  
11 print(subx4)  
11 print(subx5)
```

Output : Sarvajanik

Output : Surat

Output : INDIA

Output : Sraai olg fEgneigadTcnlg,Srt uaa,IDA

Output : AIDNI ,tarajug ,taruS ,ygonlhceT dna gnireenignE fo egelloC kinajavraS

String print format

- ❑ **str.format()** is one of the *string formatting methods* in Python3, which allows multiple substitutions and value formatting.
- ❑ This method lets us concatenate elements within a string through positional formatting.

strformat.py

```
1 x = '{} institute, Surat'
2 y = x.format('SCET')
3 print(y)
4 print(x.format('ABCD'))
```

Output : *SCET institute, Surat*

Inline function call
Output : *ABCD institute, Surat*

- ❑ We can specify multiple parameters to the function

strformat.py

```
1 x = '{} institute, {}'
2 y = x.format('SCET', 'Surat')
3 print(y)
4 print(x.format('ABCD', 'XYZ'))
```

Output : *SCET institute, Surat*

Inline function call
Output : *ABCD institute, XYZ*

String print format (cont.)

- We can specify the order of parameters in the string

strformat.py

```
1 x = '{1} institute, {0}'  
2 y = x.format('SCET', 'Surat')  
3 print(y)  
4 print(x.format('ABCD', 'XYZ'))
```

Output : *Surat institute, SCET*

Inline function call
Output : *XYZ institute, ABCD*

Data structures in python

□ There are four built-in data structures in Python - *list, dictionary, tuple and set*.

Name	Type	Description
List	<code>list</code>	Ordered Sequence of objects, will be represented with square brackets [] Example : [18, “SCET”, True, 102.3]
Dictionary	<code>dict</code>	Unordered key : value pair of objects , will be represented with curly brackets { } Example : { “college”: “SCET”, “code”: “054” }
Tuple	<code>tup</code>	Ordered immutable sequence of objects, will be represented with round brackets () Example : (18, “SCET”, True, 102.3)
Set	<code>set</code>	Unordered collection of unique objects, will be represented with the curly brackets { } Example : { 18, “SCET”, True, 102.3 }

□ Lets explore all the data structures in detail...

List

- List is a **mutable** ordered sequence of objects, **duplicate** values are allowed inside list.
- List will be represented by square brackets [].
- Python does not have array, List can be used similar to Array.

list.py	
1 my_list = ['SCET', 'institute', 'Sura']	Output : <i>institute</i> (List index starts with 0)
2 print(my_list[1])	Output : 3 (length of the List)
3 print(len(my_list))	Output : ['SCET', 'institute', 'Surat'] <i>Note : spelling of Surat is updated</i>
4 my_list[2] = "Surat"	
5 print(my_list)	
6 print(my_list[-1])	Output : <i>Surat</i> (-1 represent last element)

- We can use slicing similar to string in order to get the sub list from the list.

list.py	
1 my_list = ['SCET', 'institute', 'Surat', 'gujarat', 'INDIA']	
2 print(my_list[1:3])	Output : ['institute', 'Surat'] <i>Note : end index not included</i>

List methods

- `append()` method will add element at the end of the list.

appendlistdemo.py

```
1 my_list = ['SCET', 'institute', 'Surat']  
2 my_list.append('gujarat')  
3 print(my_list)
```

Output : ['SCET', 'institute', 'Surat', 'gujarat']

- `insert()` method will add element at the specified index in the list

insertlistdemo.py

```
1 my_list = ['SCET', 'institute', 'Surat']  
2 my_list.insert(2, 'of')  
3 my_list.insert(3, 'engineering')  
4 print(my_list)
```

Output : ['SCET', 'institute', 'of', 'engineering', 'Surat']

- `extend()` method will add one data structure (List or any) to current List

extendlistdemo.py

```
1 my_list1 = ['SCET', 'institute']  
2 my_list2 = ['Surat', 'gujarat']  
3 my_list1.extend(my_list2)  
4 print(my_list1)
```

Output : ['SCET', 'institute', 'Surat', 'gujarat']

List methods (cont.)

- `pop()` method will remove the last element from the list and return it.

poplistdemo.py

```
1 my_list = ['SCET', 'institute', 'Surat']
2 temp = my_list.pop()
3 print(temp)
4 print(my_list)
```

Output : *Surat*

Output : *['SCET', 'institute']*

- `remove()` method will remove first occurrence of specified element

removelistdemo.py

```
1 my_list = ['SCET', 'institute', 'SCET', 'Surat']
2 my_list.remove('SCET')
3 print(my_list)
```

Output : *['institute', 'SCET', 'Surat']*

- `clear()` method will remove all the elements from the List

clearlistdemo.py

```
1 my_list = ['SCET', 'institute', 'SCET', 'Surat']
2 my_list.clear()
3 print(my_list)
```

Output : *[]*

- `index()` method will return first index of the specified element.

List methods (cont.)

- `count()` method will return the number of occurrence of the specified element.

countlistdemo.py

```
1 my_list = ['SCET', 'institute', 'SCET', 'Surat']  
2 c = my_list.count('SCET')  
3 print(c)
```

Output : 2

- `reverse()` method will reverse the elements of the List

reverselistdemo.py

```
1 my_list = ['SCET', 'institute', 'Surat']  
2 my_list.reverse()  
3 print(my_list)
```

Output : ['Surat', 'institute', 'SCET']

List methods (cont.)

- ❑ '+' operator can be used to join two Lists.

concatlist.py

```
1 my_list = ['SCET', 'institute', 'Surat']  
2 mylist1 = [1,2,3]  
3 print(my_list+ mylist1 )
```

Output :
['SCET', 'institute', 'Surat', 1, 2, 3,]

- ❑ '*' operator can be used to repeat a list n times.

mullist.py

```
1 my_list = [10,20,30,40]  
2 print(my_list*3)
```

Output : [10,20,30,40,10,20,30,40,10,20,30,40]

Tuple

- Tuple is a **immutable** ordered sequence of objects, **duplicate values are allowed** inside list.
- Tuple will be represented by round brackets ().
- Tuple is similar to List but **List is mutable whereas Tuple is immutable**.

tupledemo.py

```
1 my_tuple = ('SCET', 'institute', 'of', 'engineering', 'of', 'Surat')
2 print(my_tuple)
3 print(my_tuple.index('engineering'))
4 print(my_tuple.count('of'))
5 print(my_tuple[-1])
```

Output : ('SCET', 'institute', 'of', 'engineering', 'of', 'Surat')

Output : 3 (index of 'engineering')

Output : 2

Output : Surat

Dictionary

- Dictionary is a unordered collection of key value pairs.
- Dictionary will be represented by curly brackets { }.
- Dictionary is mutable.

syntax

```
my_dict = { 'key1':'value1', 'key2':'value2' }
```

Key value is seperated by :

Key value pairs is seperated by ,

dictdemo.py

```
1 my_dict = {'college':"SCET", 'city':"Surat", 'type':"engineering"}  
2 print(my_dict['college'])  
3 print(my_dict.get('city'))
```

values can be accessed using key inside square brackets as well as using get() method

Output : SCET

Surat

Dictionary methods

- `keys()` method will return list of all the keys associated with the Dictionary.

keydemo.py

```
1 my_dict = {'college': "SCET", 'city': "Surat", 'type': "engineering"}
2 print(my_dict.keys())
```

Output : ['college', 'city', 'type']

- `values()` method will return list of all the values associated with the Dictionary.

valuedemo.py

```
1 my_dict = {'college': "SCET", 'city': "Surat", 'type': "engineering"}
2 print(my_dict.values())
```

Output : ['SCET', 'Surat', 'engineering']

- `items()` method will return list of tuples for each key value pair associated with the

itemsdemo.py

```
1 my_dict = {'college': "SCET", 'city': "Surat", 'type': "engineering"}
2 print(my_dict.items())
```

Output : [('college', 'SCET'), ('city', 'Surat'), ('type', 'engineering')]

Set

- ❑ Set is a **unordered** collection of unique objects.
- ❑ Set will be represented by curly brackets { }.

tupledemo.py

```
1 my_set = {1,1,1,2,2,5,3,9}  
2 print(my_set)
```

Output : {1, 2, 3, 5, 9}

- ❑ Set has many in-built methods such as add(), clear(), copy(), pop(), remove() etc.. which are similar to methods we have previously seen.
- ❑ Only difference between Set and List is that **Set will have only unique elements and List can have duplicate elements.**

Operators in python

- We can segregate python operators in the following groups
 - Arithmetic operators
 - Assignment operators
 - Comparison operators
 - Logical operators
 - Identity operators
 - Membership operators
 - Bitwise operators

- We will discuss some of the operators from the given list in detail in some of next slides.

Arithmetic Operators

□ Note : consider $A = 10$ and $B = 3$

Operator	Description	Example	Output
+	Addition	$A + B$	13
-	Subtraction	$A - B$	7
/	Division	A / B	3.3333333333333335
*	Multiplication	$A * B$	30
%	Modulus return the remainder	$A \% B$	1
//	Floor division returns the quotient	$A // B$	3
**	Exponentiation	$A ** B$	$10 * 10 * 10 = 1000$

Logical Operators

□ Note : consider $A = 10$ and $B = 3$

Operator	Description	Example	Output
and	Returns True if both statements are true	$A > 5$ and $B < 5$	True
or	Returns True if one of the statements is true	$A > 5$ or $B > 5$	True
not	Negate the result, returns True if the result is False	not ($A > 5$)	False

Identity & Member Operators

□ Identity Operator

□ Note : consider $A = [1,2]$, $B = [1,2]$ and $C=A$

Operator	Description	Example	Output
is	Returns True if both variables are the same object	A is B A is C	FALSE TRUE
is not	Returns True if both variables are different object	A is not B	TRUE

□ Member Operator

□ Note : consider $A = 2$ and $B = [1,2,3]$

Operator	Description	Example	Output
in	Returns True if a sequence with the specified value is present in the object	A in B	TRUE
not in	Returns True if a sequence with the specified value is not present in the object	A not in B	FALSE

Operator Precedence

□ Precedence represents the priority level.

Precedence	Operator Sign	Operator Name
Highest	**	Exponentiation
	+X, -X, ~X	Unary positive, unary negative, bitwise negation
	*, /, //, %	Multiplication, division, floor, division, modulus
	+, -	Addition, subtraction
	<<, >>	Left-shift, right-shift
	&	Bitwise AND
	^	Bitwise XOR
		Bitwise OR
	==, !=, <, <=, >, >=, is, is not	Comparison, Identity
	not	Boolean NOT
	and	Boolean AND
Lowest	or	Boolean OR

Mathematical Function

□ In program

```
import math
x=math.sqrt(16)
print(x)
```

□ This will display 4.

Function	Returns (Description)
abs(x)	The absolute value of x: the (positive) distance between x and zero.
ceil(x)	The ceiling of x: the smallest integer not less than x. math module is required.
cmp(x, y)	-1 if x < y, 0 if x == y, or 1 if x > y. Deprecated in Python 3; Instead use return (x>y)-(x<y) .
exp(x)	The exponential of x: e^x math module is required.
fabs(x)	The absolute value of x. math module is required.
floor(x)	The floor of x: the largest integer not greater than x. math module is required.
log(x)	The natural logarithm of x, for x > 0. math module is required.
log10(x)	The base-10 logarithm of x for x > 0. math module is required.
max(x1, x2,...)	The largest of its arguments: the value closest to positive infinity.
min(x1, x2,...)	The smallest of its arguments: the value closest to negative infinity.
modf(x)	The fractional and integer parts of x in a two-item tuple. Both parts have the same sign as x. The integer part is returned as a float. math module is required.
pow(x, y)	The value of x^y .
round(x [,n])	x rounded to n digits from the decimal point. Python rounds away from zero as a tie-breaker: round(0.5) is 1.0 and round(-0.5) is -1.0.
sqrt(x)	The square root of x for x > 0. math module is required.